

Resumo Executivo do Passo 1.3: Implantação e orquestração do ferramental de qualidade de código de altíssima performance. O projeto passa a contar com o **Ruff** (motor em Rust que unifica linter e formatação) e o **Pytest 9.1** integrado ao **Pytest-Cov** para testes automatizados e métricas de cobertura. Todas as regras foram consolidadas de forma declarativa no `pyproject.toml` sob os padrões **PEP 518**, **PEP 621** e **PEP 735**, com geração de lockfile determinístico (`uv.lock`) e validação operacional em menos de 50ms.

1. Matriz de Entregas por Micro-Fase

Micro-Fase	Escopo & Atribuição	Detalhamento da Implementação Técnica	Avaliação
1.3.1	Instalação de Dependências	Instalação isolada via <code>uv pip install</code> de <code>ruff==0.16.8</code> , <code>pytest==9.1.1</code> e <code>pytest-cov==7.1.0</code> . Resolução do grafo em 686ms e instalação em 1.76s. Zero poluição no host.	✓ Aprovado
1.3.2	<code>pyproject.toml</code> & <code>uv.lock</code>	Estruturação dos metadados (PEP 621), dependências de desenvolvimento (PEP 735), restrição <code>requires-python = ">=3.12,<3.13"</code> e <code>[tool.uv] package = false</code> . Lockfile multiplataforma gerado em 14ms.	✓ Aprovado
1.3.3	Configuração do Ruff	Regras ativas: E, W, F, I (isort), B (bugbear), C4 (comprehensions), UP (pyupgrade), ARG (unused args), SIM (simplify). Target <code>py312</code> , 88 caracteres, aspas duplas e imports first-party mapeados.	✓ Aprovado
1.3.4	Configuração do Pytest & Testes	Configurado <code>[tool.pytest.ini_options]</code> com <code>--strict-markers</code> , descoberta em <code>tests/</code> e taxonomia SCSi (unit, integration, e2e, celery, ai). Criada a suíte <code>test_harness.py</code> com 3/3 testes aprovados.	✓ Aprovado
1.3.5	Smoke Test & Relatório PDF	Execução encadeada do pipeline de qualidade: <code>ruff check</code> . limpo, formatação validada e suíte de testes executada em 0.03s. Emissão do relatório executivo e consolidação documental.	✓ Aprovado

2. Arquitetura do Harness de Qualidade

 **Linter & Formatter: Ruff 0.16.8**

- Motor:** Compilado nativamente em Rust, executando em espaço de usuário dentro de `.venv\Scripts\ruff.exe`.
- Performance:** Análise estática do projeto completo em **15ms** (10x a 100x mais veloz que `flake8/black`).
- Organização de Imports:** Algoritmo determinístico `isort` separando Standard Library, Third-Party (Django/Celery) e First-Party (`core`, `apps`, `config`).
- Modernizador Automático:** Regra `UP` audita sintaxe obsoleta e converte para Python 3.12 nativo.

 **Framework de Testes: Pytest 9.1 & Cobertura**

- Strict Mode:** `--strict-markers` proíbe marcadores não mapeados, eliminando falhas silenciosas por erros de digitação.
- Taxonomia Semântica:** Separação expressa entre testes unitários rápidos e testes de integração com containers.
- Branch Coverage:** Monitoramento com `coverage.py` analisando todos os ramos condicionais (`if/else`).
- Assert Rewriting:** Diagnóstico enriquecido de variáveis em tempo de execução nas falhas de teste.

3. Evidência do Pipeline de Qualidade Executado

```
PS C:\estudo_arquitetura_sistemas_pycoder> uv run ruff check . All checks passed! PS
C:\estudo_arquitetura_sistemas_pycoder> uv run ruff format --check . 5 files already formatted PS
C:\estudo_arquitetura_sistemas_pycoder> uv run pytest ===== test session starts
===== platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 rootdir:
C:\...\estudo_arquitetura_sistemas_pycoder, configfile: pyproject.toml testpaths: tests, plugins: cov-7.1.0
collected 3 items tests/test_harness.py::test_python_runtime_version PASSED [ 33%]
tests/test_harness.py::test_virtualenv_isolation PASSED [ 66%]
tests/test_harness.py::test_python_312_syntax_features PASSED [100%] ===== 3 passed in
0.03s =====
```

4. Decisões Arquiteturais Consolidadas (ADRs)

Decisão Técnica	Racional Arquitetural	Impacto no Deploy / Produção
Modo Aplicação (tool.uv package=false)	Sistemas Django/Celery são aplicações web e não pacotes distribuíveis no PyPI. Evita overhead de build de wheels.	Deploy via Dockerfile simplificado: <code>uv sync --frozen --no-dev</code> sem dependências de compilação C/Rust.
Teto de Runtime (<3.13)	Impede o uso inadvertido de versões preliminares (como Python 3.14 do host) que quebram pacotes compilados.	Garante paridade 1:1 rigorosa com os containers oficiais <code>python:3.12-slim</code> que rodarão na VPS Hostinger.
Lockfile Multiplataforma (uv.lock)	Contém hashes SHA-256 calculados simultaneamente para Windows x86_64, Linux glibc x86_64 e Linux aarch64.	Na VPS Ubuntu, <code>uv sync --frozen</code> nunca re-resolve pacotes; baixa exatamente as mesmas versões auditadas localmente.